

To design an optimal TSDB, we must analyse the properties of time series data, and the demands of time series analysis applications. The typical characteristics of time series data and its use cases are:

- Time series is a sequence of values, each with a time value indicating when the value was recorded.
- Time series data entries are rarely amended.
- Time series data is often retrieved by reading a contiguous sequence of samples.
- Most of the time, we collect and store multiple time series. Queries to retrieve data from one or a few time series for a particular time range are very common.
- The volume and velocity of time series data is very high.
- Both long-term and short-term trends in the time series are very important for analysis.
- Summarising or aggregating high volume time series data sets is a very basic requirement.
- Traditional DB operations such as searching, sorting, joining tables, etc, are not required.

Properties that make time series data very different from other data workloads are data life cycle management, summarisation, and large range scans of many records. TSDB is designed to simplify and strengthen the process for real-world time series applications.

Storing time series data in flat files limits its utility. Data will outgrow these and the access is inefficient. Traditional RDBMS databases are not designed from the ground up for time series data storage. They will not scale well to handle huge volumes of time series data. Also, the schema is not appropriate. Getting a good performance for time series from an SQL database requires significant customisation and configuration. Without that, unless you're working with a very small data set, an SQL-based database will simply not work properly. A NoSQL non-relational database is preferred because it scales well and efficiently to enable rapid queries based on time range.

A TSDB is optimised for best performance for queries based on a range of time. New NoSQL non-relational databases come with considerable advantages (like flexibility and performance) over traditional relational databases (RDBMS) for this purpose. NoSQL databases and relational databases share the same basic goals: to store and retrieve data and to coordinate changes. The difference is that NoSQL databases trade away some of the capabilities of relational databases in order to improve scalability. The benefits of making this trade include greater simplicity in the NoSQL database, the ability to handle semi-structured and denormalised data and, potentially, much higher scalability for the system.

At very large scales, time-based queries can be implemented as large, contiguous scans that are very efficient if the data is stored appropriately in a time series database. And if the amount of data is very large, a non-relational TSDB in a NoSQL system is typically needed to provide sufficient scalability.

Non-relational time series databases enable discovery of patterns in time series data, long-term trends, and correlations between data representing different types of events. The time ranges of interest extend in both directions. In addition to the very short time-range queries, long-term histories for time series data are needed, especially to discover complex trends.

Time series databases have key architectural design properties that make them very different from other databases. These include time-stamp data storage and compression, data life cycle management, data summarisation, the ability to handle large time series-dependent scans of many records, and time series aware queries.

For example, with a time series database, it is common to request a summary of data over a large time period. This requires going over a range of data points to perform computations like a percentile increase this month of a metric over the same period in the last six months, summarised by month. This kind of workload is very difficult to optimise for with a distributed key value store. TSDBs are optimised for exactly this use case and can give millisecond-level responses over months of data. Here is another example. With time series databases, it's common to keep high precision data around for a short period of time. This data is aggregated and down sampled into long-term trend data. This means that for every data point that goes into the database, it will have to be deleted after its period of time is up. This kind of data life cycle management is difficult for application developers to implement in regular databases. They must devise schemes

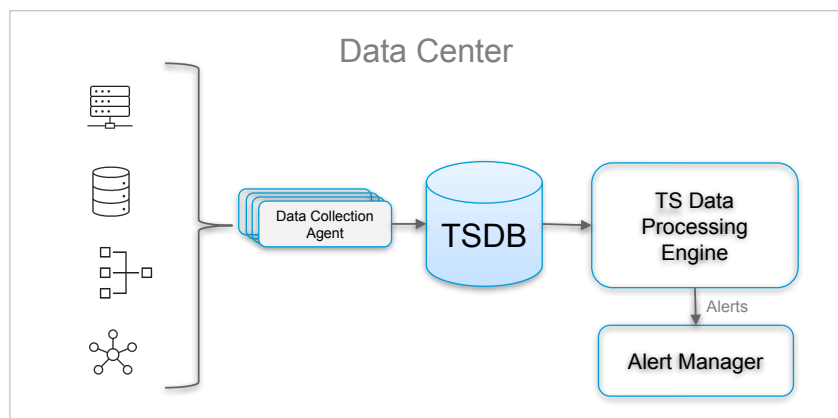


Figure 1: Alerting engine based on time series data analysis